Formalisms Every Computer Scientist Should Know: Homework Solutions

1 Homework 1

Theorem 1.1. *If there is a one-to-one function on a set A to a subset of a set B and there is also a one-to-one function on B to a subset of A, then A and B are equipollent.*

Proof. We need to show that there exists a bijection between A and B.

Assume such two functions exist. Fix the two functions $\widehat{f} : A \to B$ and $\widehat{g} : B \to A$ be one-to-one functions.

Then there exist bijections $\widehat{f}^{-1} : \widehat{f}(A) \to A$ and $\widehat{g}^{-1} : g(B) \to B$.

Define $A_C \subseteq A$ as the set of such elements $a \in A$ that $(\widehat{g} \circ \widehat{f})^n a = a$ for some integer $n \in \mathbb{N}$ (zero is not in $\mathbb{N}$).

In the same way, define $B_C = \{b \in B \mid \exists n \in \mathbb{N} : (\widehat{f} \circ \widehat{g})^n b = b\}$.

For a given element $a \in A$, define a sequence of ancestors $X(a) = (a, \widehat{g}^{-1}(a), \widehat{f}^{-1}(\widehat{g}^{-1}(a)), \ldots)$. The sequence is finite if the last element is in $A \setminus \widehat{g}(B)$ or in $B \setminus \widehat{f}(A)$, and infinite otherwise. In the same way, for $b \in B$, define $X(b) = (b, \widehat{f}^{-1}(b), g^{-1}(\widehat{f}^{-1}(b)), \ldots)$.

Define $A_I = \{a \in A \mid X(a) \text{ is infinite}\}$, $A_E = \{a \in A \mid |X(a)| \text{ is even}\}$, $A_O = \{a \in A \mid |X(a)| \text{ is odd}\}$. Observe that A_I, A_E, A_O are disjoint and cover A. Similarly, define B_I, B_E, B_O.

Notice that $A_C \subseteq A_I$, since for any $a \in A_C$, $X(a)$ is periodic and infinite.

I claim that $\widehat{f}$ maps A_I bijectively onto B_I. To see this, we need to show that for any $b \in B_I$, there exists a preimage $a \in A_I$ such that $\widehat{f}(a) = b$. Such a is the second element of $X(b)$, $a = \widehat{f}^{-1}(b)$. Then $\widehat{f}(a) = b$. a is indeed in A_I, because $X(a) = (a = \widehat{f}^{-1}(b), \widehat{g}^{-1}(\widehat{f}^{-1}(b)), \ldots)$ is also infinite.

Next, show that f maps A_E bijectively onto B_O. To see this, we need to show that for any $b \in B_O$, there exists a preimage $a \in A_E$ such that $\widehat{f}(a) = b$. Such a is the second element of $X(b)$, $a = \widehat{f}^{-1}(b)$. Then $\widehat{f}(a) = b$. a is indeed in A_E, because $X(a) = (a = \widehat{f}^{-1}(b), \widehat{g}^{-1}(\widehat{f}^{-1}(b)), \ldots)$. Then $|X(a)| = |X(b)| - 1$ is even.

In the same way, g maps B_E bijectively onto A_O. Therefore, g^{-1} maps A_O bijectively onto B_E.

Define a map $h : A \to B$:

$$h(a) = \begin{cases} \widehat{f}(a), & a \in A_I \\ \widehat{f}(a), & a \in A_E \\ \widehat{g}^{-1}(a), & a \in A_O \end{cases}, \tag{1}$$

which is the desired bijection between A and B. □

2 Homework 2

Theorem 2.1 (Knaster-Tarski)**.** *Let $(A, \sqsubseteq)$ be a complete lattice and let F be a monotone function on A. Further, let $\hat{y} = \bigsqcup\{x \mid x \sqsubseteq F(x)\}$ and $\hat{z} = \bigsqcap\{x \mid F(x) \sqsubseteq x\}$. It holds, that:*

1. *$\hat{y}$ and $\hat{z}$ are fixpoints of F,*
2. *for all fixpoints x of F, $\hat{z} \sqsubseteq x \sqsubseteq \hat{y}$*

Proof. Suppose $(A, \sqsubseteq)$ is a complete lattice and let F be a monotone function on A. Let U be the set of elements $x \in A$ for which $x \sqsubseteq F(x)$ and let D be the set of elements $x \in A$ for which $x \sqsupseteq F(x)$. Then as described in the lecture let us denote the join and meet of these two sets respectively as $\hat{y} = \bigsqcup U$ and $\hat{z} = \bigsqcap D$ (this meet and join exist because the considered lattice is complete). We shall prove:

1. $\hat{y}$ and $\hat{z}$ are fixpoints of F.
2. For all fixpoints x of F the following holds $\hat{z} \sqsubseteq x \sqsubseteq \hat{y}$.

1. Let us first prove that $\hat{y}$ is a fixpoint of F. Let us pick an arbitrary element $u \in U$, then by definition of $\hat{y}$ we have $u \sqsubseteq \hat{y}$. Because F is monotone this implies $F(u) \sqsubseteq F(\hat{y})$. And because u was picked arbitrarily from U this means that $F(\hat{y})$ is an upper bound on U. But $\hat{y}$ is the least upper bound of U and thus $\hat{y} \sqsubseteq f(\hat{y})$. Again by applying the monotonicity of F we get $F(\hat{y}) \sqsubseteq F(F(\hat{y}))$ and we can conclude $F(\hat{y}) \in U$. But because $\hat{y}$ is the join of U and $F(\hat{y})$ is an element of U it follows $F(\hat{y}) \sqsubseteq \hat{y}$. Together with the inequality $\hat{y} \sqsubseteq F(\hat{y})$ this gives us $F(\hat{y}) = \hat{y}$, hence $\hat{y}$ is a fixpoint of F. The proof that $\hat{z}$ is a fixpoint of F can be done along the same lines by considering $\sqsupseteq$ instead of $\sqsubseteq$ and the set D instead of U.

2. Suppose $\hat{x}$ is a fixpoint of F. Then clearly $\hat{x} \in U$ and hence $\hat{x} \sqsubseteq \hat{y}$ as $\hat{y}$ is the join of U. Similarly $\hat{x} \in D$ and hence $\hat{x} \sqsupseteq \hat{z}$ as $\hat{z}$ is the meet of D. So it follows that $\hat{z} \sqsubseteq \hat{x} \sqsubseteq \hat{y}$ and the proof is done. □

3 Homework 3

Theorem 3.1. *Let F be a monotone function over a complete lattice $(A, \sqsubseteq)$. Then for every non-decreasing sequence of elements $x_0, x_1, \ldots$, we have that $F(\bigsqcup\{x_0, x_1, \ldots\}) \sqsupseteq \bigsqcup\{F(x_i) \mid i \in \mathbb{N}\}$.*

Proof. We write $X = F(\bigsqcup\{x_0, x_1, \ldots\})$ and $Y = \bigsqcup\{F(x_i) \mid i \in \mathbb{N}\}$. Let us assume the contrary, i.e., that $X \sqsubset Y$. Therefore, there has to be $i \in \mathbb{N}$ such that $F(x_i) \not\sqsubseteq X$. Otherwise X would be an upper bound of $\{F(x_i) \mid i \in \mathbb{N}\}$ which would violate the definition of a least upper bound of Y. From the definition of an upper bound, we know that $x_i \sqsubseteq \bigsqcup\{x_0, x_1, \ldots\}$. If we apply monotonicity to the last inequality, we get $F(x_i) \sqsubseteq F(\bigsqcup\{x_0, x_1, \ldots\}) = X$ which is a contradiction. □

Remark 3.1. It is not true that for any monotone function F over a complete lattice $(A, \sqsubseteq)$ and every non-decreasing sequence of elements $x_0, x_1, \ldots$, we have $F(\bigsqcup\{x_0, x_1, \ldots\}) \sqsubseteq \bigsqcup\{F(x_i) \mid i \in \mathbb{N}\}$.

Proof. Let us take the set $A = \mathbb{N} \cup \infty_1 \cup \infty_2$ and $\sqsubseteq$ defined naturally for $\mathbb{N}$ and $\mathbb{N} \sqsubset \infty_1 \sqsubset \infty_2$. This is a complete lattice. We define a monotone function F as: $F(x) = \infty_1$ for every $x \in \mathbb{N}$, $F(\infty_1) = F(\infty_2) = \infty_2$. If we define a nondecreasing sequence $x_0, x_1, \ldots = \mathbb{N}$, then

$$F\left(\bigsqcup\{x_0, x_1, \ldots\}\right) = F(\infty_1) = \infty_2 \not\sqsubseteq \infty_1 = \bigsqcup\{\infty_1\} = \bigsqcup\{F(x_i) \mid i \in \mathbb{N}\}.$$

□

4 Homework 4

4.1 Sorting

Definition 4.1 (`ordered`)**.** A function $\texttt{ordered} : \Sigma^* \to \{true, false\}$ is defined as follows. $\texttt{ordered}(\varepsilon) = true$, $\texttt{ordered}(a) = true$, $\texttt{ordered}(abx) = (a \leq b) \wedge \texttt{sorted}(bx)$, where $a \in \Sigma$, $x \in \Sigma^*$.

Lemma 4.1. $\texttt{sorted}(\texttt{ms}(x)) = true$.

Proof. We will use induction on the length of x. Obviously, $\texttt{sorted}(\texttt{ms}(\varepsilon)) = \texttt{sorted}(\texttt{ms}(a)) = true$.

Recall that $\texttt{ms}(x) = \texttt{merge}(\texttt{ms}(\texttt{odd}(x)), \texttt{ms}(\texttt{even}(x)))$. By the induction assumption, we know that $\texttt{ms}(\texttt{odd}(x)), \texttt{ms}(\texttt{even}(x))$ are sorted, since their length is less than the length of x. Now it is enough to show that $\texttt{sorted}(\texttt{merge}(y, z)) = true$ if y and z are sorted. Consider $\texttt{sorted}(\texttt{merge}(ay, bz))$. Without loss of generality, let $a \leq b$, so $\texttt{sorted}(\texttt{merge}(ay, bz)) = \texttt{sorted}(a\texttt{merge}(y, bz)) = true$, since a is less or equal to both b and the next element of y, and $\texttt{sorted}(\texttt{merge}(y, bz)) = true$ holds by induction. □

Definition 4.2 (permutation)**.** For all natural n, the function permutation : $\Sigma^n \times \Sigma^n \to \{true, false\}$ is defined as follows. $\texttt{permutation}(\varepsilon, \varepsilon) = true$, $\texttt{permutation}(a, b) = (a == b)$, $\texttt{permutation}(ax, yaz) = \texttt{permutation}(x, yz)$, and if $a \notin y$ then $\texttt{permutation}(ax, y) = false$, where $a, b \in \Sigma$, $x, y, z \in \Sigma^*$.

Lemma 4.2. $\texttt{permutation}(x, \texttt{ms}(x)) = true$.

Proof. We will use induction on the length of x. Obviously, $\texttt{permutation}(\varepsilon, \texttt{ms}(\varepsilon)) = \texttt{permutation}(a, \texttt{ms}(a)) = true$.

First, let's show that $\texttt{merge}(\texttt{x}, \texttt{y})$ is a permutation of xy. Indeed, if, for instance, $a \leq b$, then $\texttt{permutation}(\texttt{merge}(ax, by), axby) = \texttt{permutation}(a\texttt{merge}(x, by), axby) = \texttt{permutation}(\texttt{merge}(x, by), xby) = true$ by induction.

Finally, $\texttt{ms}(x) = \texttt{merge}(\texttt{ms}(\texttt{odd}(x)), \texttt{ms}(\texttt{even}(x)))$. By the induction assumption, we know that $\texttt{ms}(\texttt{odd}(x)), \texttt{ms}(\texttt{even}(x))$ are permutations of $\texttt{odd}(x), \texttt{even}(x)$ respectively, since their length is less than the length of x. Moreover, as we have already shown above, $\texttt{merge}(\texttt{ms}(\texttt{odd}(x)), \texttt{ms}(\texttt{even}(x)))$ is a permutation of $\texttt{ms}(\texttt{odd}(x))\texttt{ms}(\texttt{even}(x))$, which is a permutation of $\texttt{odd}(x)\texttt{even}(x)$. Finally, it is sufficient to prove that $\texttt{odd}(x)\texttt{even}(x)$ is a permutation of x. Using induction, $\texttt{permutation}(\texttt{odd}(ax)\texttt{even}(ax), ax) = \texttt{permutation}(a\texttt{even}(x)\texttt{odd}(x), ax) = \texttt{permutation}(\texttt{even}(x)\texttt{odd}(x), x) = \texttt{permutation}(\texttt{odd}(x)\texttt{even}(x), x) = true$. Here, we used the induction hypothesis and the fact that xy is a permutation of yx (also straightforwardly proved with induction, if needed). □

4.2 The human and monkeys theorem

Let's have a relation $<$, defined as follows: $x < y$ if $\texttt{parent}(x, y)$ or there exists z such that $x < z$ and $\texttt{parent}(z, y)$. We will assume that this relation is well-founded. Also, for all x, either $\texttt{human}(x)$ or $\texttt{monkey}(x)$.

Theorem 4.3. *If $x < y$, and $\texttt{human}(y)$, and $\texttt{monkey}(x)$, then there exist z_1, z_2 such that $\texttt{parent}(z_1, z_2)$, and $\texttt{human}(z_2)$, and $\texttt{monkey}(z_1)$.*

Proof. Define a sequence $w_1, w_2, \ldots$ inductively like this: $w_1 = y$, w_{n+1} is such that $\texttt{parent}(w_{n+1}, w_n)$, and $x < w_{n+1}$. If $w_n = x$, then terminate the sequence. Obviously, $w_1 > w_2 > \ldots$. Since $<$ is well-founded, the sequence has to be finite: $y = w_1 > w_2 > \ldots > w_k = x$.

For the sake of contradiction, suppose, there is no such i, that $\texttt{parent}(w_{i+1}, w_i)$, and $\texttt{human}(w_i)$, and $\texttt{monkey}(w_{i+1})$. Then, by induction, all w_i are humans. This contradicts the fact that w_k is a monkey. Therefore, the desired z_1, z_2 exist among some w_{i+1}, w_i, as requested. □

5 Homework 6: Soundness of FOL Resolution

In the following we use sets for both representing formulas in CNF and individual clauses. Hence, a formula in CNF is a set containing sets of literals.

We prove the soundness of the following FOL resolution rule:

$$\frac{C_1[l_1] \qquad C_2[\neg l_2]}{C_1\theta[\bot] \vee C_2\theta[\bot]} \; l_1\theta = l_2\theta$$

Lemma 5.1. *For any $C_1[l]$, $C_2[\neg l]$ first-order clauses and $\mathcal{I}$ interpretation, we get*

$$\mathcal{I} \models \{C_1[l], C_2[\neg l]\} \Rightarrow \mathcal{I} \models C_1[\bot] \vee C_2[\bot].$$

Proof. Let $\mathcal{I}$, $C_1[l]$, $C_2[\neg l]$ be arbitrary and $\mathcal{I} \models C_1[l]$ and $\mathcal{I} \models C_2[\neg l]$. We show $C_1[\bot] \vee C_2[\bot]$. Consider the two cases for $[\![l]\!]_{\mathcal{I}}$:

Case 1. $[\![l]\!]_{\mathcal{I}} = true$ **:** Consider the clause $C_2' = C_2 \setminus \{\neg l\}$, corresponding to the clause C_2 with $\neg l$ removed. Since $[\![\neg l]\!]_{\mathcal{I}} = false$, and $\mathcal{I} \models C_2[\neg l]$, $\mathcal{I} \models C_2'$. Moreover, since $\mathcal{I} \models C_2'$, $\mathcal{I} \models C_2' \cup \{\bot\}$ and therefore $\mathcal{I} \models C_2[\bot]$. Finally, since $\mathcal{I} \models C_2[\bot]$, $\mathcal{I} \models C_1[\bot] \vee C_2[\bot]$.

Case 2. $[\![l]\!]_{\mathcal{I}} = false$ **:** symmetric to Case 1. □

Lemma 5.2. *Let $\mathcal{I}$, C be an arbitrary interpretation and clause, and σ a substitution.*

$$\mathcal{I} \models \forall x_1 ... x_n, C \Rightarrow \mathcal{I} \models \forall x_1 ... x_n, C\sigma$$

Proof. Let $\mathcal{I}, C, \sigma$ be arbitrary with $\mathcal{I} \models \forall x_1 ... x_n C$, and assume towards a contradiction that $\mathcal{I} \not\models \forall x_1 ... x_n, C\sigma$. By the definition of $[\![\forall x.\phi]\!]$, there must be a mapping $\kappa = [y_1 \to d_1, ..., y_n \to d_n]$ from free variables in $C\sigma$ to $d_1 ... d_n \in D_{\mathcal{I}}$ s.t. $[\![C\sigma]\!]_{\mathcal{I}[\kappa]} = \text{false}$.
We now construct a mapping $\kappa' = x_1 \to d_1', ..., x_n \to d_n'$ s.t. $[\![C]\!]_{\mathcal{I}[\kappa']} = false$. Consider the syntactic substitution $\sigma = x_1 \to t_1, ..., x_n \to t_n$, we simply define $\kappa' = [x_1 \to t_1\kappa, ..., x_n \to t_n\kappa]$. Hence, κ' is a map from free variables in C to elements in the domain $D_{\mathcal{I}}$ and is a valid mapping to close the formula C. Moreover, $[\![C]\!]_{\mathcal{I}[\kappa']} = [\![C\sigma]\!]_{\mathcal{I}[\kappa]} = false$. Hence, $\mathcal{I} \not\models \forall x_1, ..., x_n C$, a contradiction. □

Theorem 5.3 (Soundness of FOL resolution)**.**
For any clause set S, clauses $C_1[l_1] \in S$, $C_2[\neg l_2] \in S$, and substitution θ s.t. $l_1\theta = l_2\theta$: S is satisfiable iff $S \cup \{C_1\theta[\bot] \vee C_2\theta[\bot]\}$ is satisfiable.

Proof.

$\Rightarrow$ Let S, $C_1[l_1] \in S$, $C_2[\neg l_2] \in S$ s.t. $l_1\theta = l_2\theta$, be arbitrary and S satisfiable. By definition of satisfiable, let $\mathcal{I}$ s.t. $\mathcal{I} \models S$ and therefore $\mathcal{I} \models C_1$, and $\mathcal{I} \models C_2$. Note, that both clauses C_1 and C_2 are implicitely universally quantified, and since $\forall$ distributes over $\wedge$, we get $\mathcal{I} \models \forall x_1 ... x_n, C_1$ and $\mathcal{I} \models \forall x_1 ... x_n, C_2$ where $x_1 ... x_n$ are the free variables in S.
By Lemma 5.2, $\mathcal{I} \models \forall x_1 ... x_n, C_1\theta$ and $\mathcal{I} \models \forall x_1 ... x_n, C_2\theta$. In the following we drop the implicitely quantified variables again. Let $l = l_1\theta = l_2\theta$ and since the order of substitution is irrelevant, $\mathcal{I} \models C_1[l]\theta$ and $\mathcal{I} \models C_2[l]\theta$. By Lemma 5.1, $\mathcal{I} \models C_1\theta[\bot] \vee C_2\theta[\bot]$, and therefore, by definition of satisfiable, $S \cup \{C_1\theta[\bot] \vee C_2\theta[\bot]\}$ is satisfiable.

$\Leftarrow$ Let $S, C_1[l_1] \in S, C_2[\neg l_2] \in S$ s.t. $l_1\theta = l_2\theta$, be arbitrary and $S \cup \{C_1\theta[\bot] \vee C_2\theta[\bot]\}$ satisifable. By definition of satisfiable, let $\mathcal{I}$ s.t. $\mathcal{I} \models S \cup \{C_1\theta[\bot] \vee C_2\theta[\bot]\}$. By definition of $\wedge$, $\mathcal{I} \models S$ and therefore S is satisfiable. $\square$

6 Homework 7

An instance of PCP is a finite set of two-part tiles $\frac{s_1}{t_1}, \frac{s_2}{t_2}, \ldots, \frac{s_k}{t_k}$ where each tile is from $\frac{\{0,1\}^*}{\{0,1\}^*}$. There exists a solution to a PCP instance if there is a finite indexing $i_1, i_2, \ldots, i_l \in 1, \ldots, k$ such that $s_{i_1} s_{i_2} \ldots s_{i_l} = t_{i_1} t_{i_2} \ldots t_{i_l}$.

Theorem 6.1. *Solving PCP is undecidable.*

By reducing the problem of PCP to a statement of first order logic, we prove its undecidability. For a PCP instance, let us define a signature:

- Constant symbol e.
- Unary function symbols f_0 and f_1.
- Binary predicate symbol p.

We will use unary function symbols $s_1, \ldots, s_k, t_1, \ldots, t_k$ as abbreviations for appending a tile to a string (e.g., if $s_i = 010$, then $s_i = f_0(f_1(f_0(\ldots)))$). Let us define the following axioms:

$$\varphi_1 := \bigwedge_{i \in 1,\ldots,k} p(s_i(e), t_i(e))$$

$$\varphi_2 := \forall u, v \left(p(u, v) \Rightarrow \bigwedge_{i \in 1,\ldots,k} p(s_i(u), t_i(v)) \right)$$

$$\varphi_3 := \exists z \; p(z, z)$$

Theorem 6.2. *Deciding validity of a first order formula is undecidable.*

Proof. We reduce a PCP instance to deciding validity of the statement $(\varphi_1 \wedge \varphi_2) \Rightarrow \varphi_3$. We need to prove that the formula is valid iff there is a solution to the PCP instance.

$\Leftarrow$ Let $i_1, i_2, \ldots, i_l \in 1, \ldots, k$ be indices of the solution of a PCP instance. Hence, $s_{i_1} s_{i_2} \ldots s_{i_l} = t_{i_1} t_{i_2} \ldots t_{i_l}$. We need to show the validity of the formula, i.e., that it holds in any interpretation. The above fact implies that

$$u_s = s_k(s_{i_{k-1}}(\cdots(s_{i_1}(e))\cdots)) = t_{i_k}(t_{i_{k-1}}(\cdots(t_{i_1}(e))\cdots)) = u_t$$

(note that this holds syntactically because these functions are just abbreviations for f_0, f_1).

Inductively, on the length of k, we prove that $p(u_s, u_t)$ assuming φ_1 and φ_2.

Base case: If $k = 1$. Using the statement φ_1, we know that $p(s_{i_1}(e), t_{i_1}(e))$ which is what we want.

Inductive case: We know that $p(s_{k-1}(\cdots(s_{i_1}(e))\cdots), t_{k-1}(\cdots(t_{i_1}(e))\cdots))$. Then, using φ_2, we know that this implies $p(s_k(s_{i_{k-1}}(\cdots(s_{i_1}(e))\cdots)), t_{i_k}(t_{i_{k-1}}(\cdots(t_{i_1}(e))\cdots)))$.

Hence, we proved $p(u_s, u_t)$ and since $u_s = u_t$ we get that φ_3. Therefore, the formula is valid.

$\Rightarrow$ We assume validity of the formula and find a solution for the PCP instance. Therefore, we may choose any interpretation.

Let us choose the following: Domain is the set of finite strings over $\{0,1\}^*$. Symbol e is the empty string. Unary functions f_0 and f_1 represent appending 0 or 1 to a string, respectively. Finally, the subset of $\{0,1\}^* \times \{0,1\}^*$ for which the predicate p is valid is chosen to be minimal in terms of inclusion while preserving validity of φ_1 and φ_2, i.e., we interpret $p(u, v)$ as there is a sequence of tiles such that u is made up of the upper parts of the tiles and v is made up of the bottom parts of the tiles.

Such intepretation exists. Therefore, the statement φ_3 is true which means there is a finite string z such that $p(z, z)$. By well-founded induction on the length of z, we find the solution to the PCP instance.

Since p is minimal up to inclusion closed under φ_1 and φ_2, then there are u, v for each $p(u, v)$ such that u, v either match the formula φ_1 or right-hand side of the formula φ_2.

Let us assume that z matches the first axiom, i.e., there is $i \in 1, \ldots, k$ such that $z = s_i(e) = t_i(e)$. Then, we found a solution to the problem which is using the tile i.

Let us now assume that z matches the second axiom, i.e., there is $u, v \in \{0,1\}^*$ and $i \in 1, \ldots, k$ such that $z = s_i(u) = t_i(v)$ and $p(u, v)$. This means the last tile used was i and the rest of the tiling is obtained using the induction hypothesis from u, v.

Therefore, we obtained a finite solution to the PCP instance.

□

Theorem 6.3. *A theory T is consistent iff T does not contain all wffs over the T-signature.*

Proof. If T does contain all wffs over the T-signature, then there is no model of T since for every model $\mathcal{I}$, either φ or $\neg\varphi$ is not true (where φ is any wff), i.e., either $\mathcal{I} \nvDash \varphi$ or $\mathcal{I} \nvDash \neg\varphi$ which means $\mathcal{I} \nvDash T$. Non-existence of a model implies inconsistency directly from the definition.

Let us assume T is inconsistent and let ψ be any wff. Let $\mathcal{I}$ be a model. Since T is inconsistent, then $\mathcal{I} \nvDash T$ which means there is a formula $\varphi \in T$ such that $\mathcal{I} \nvDash \varphi$. Therefore, the statement $\forall\mathcal{I}((\forall\xi \in T, \mathcal{I} \vDash \xi) \Rightarrow \mathcal{I} \vDash \psi)$ is true since the implication is vacuously true. Hence, directly from the definition, ψ is in T which means any wff is in T. □

7 Homework 8: Congruence Closure Algorithm and Complexity

In this assignment, we will demonstrate that the runtime of the congruence closure implemented using a *union-find* data structure is in $\mathcal{O}(n \log n)$. To this end, we will first introduce the union-find data structure and its operations and then give a precise description of the congruence closure algorithm, analyzing each step in terms of its complexity.

7.1 Union-Find Data Structure

The union-find data structure, also known as the disjoint-set data structure, efficiently maintains a collection of disjoint sets. Each set is represented as a tree, with each node pointing to its parent. The root of each tree is the so-called *representative* of that set.

The union-find data structure supports two main operations:

- `find`(x): Determines the representative of the set containing the element x.
- `union`(x, y): Merges the sets with representatives x and y into a single set.

7.1.1 Optimizations

To improve the efficiency of union-find operations, one can employ two standard heuristics:

- **Path Compression**: During a `find` operation, we make each node on the search path point directly to the root. This flattens the structure of the tree, leading to faster subsequent `find` operations.
- **Union by Rank (or Size)**: Let the *rank* of a node in the union find structure denote an upper bound on the height of the subtree rooted at that node. When performing a `union` operation, we attach the tree with the smaller rank (or size) to the root of the tree with the larger rank. This prevents the trees from becoming too tall.

7.1.2 Time Complexity

With these optimizations, both the `find` and `union` operations have an amortized time complexity of $\mathcal{O}(\alpha(n))$, where $\alpha(n)$ is the inverse Ackermann function. The function $\alpha(n)$ grows extremely slowly and can be considered a constant (less than 5) for all practical values of n.

As this exercise assumes that the union-find data structure is available, we will not give a detailed proof of the time complexity of the operations.

7.2 Congruence Closure Algorithm

The congruence closure algorithm computes the smallest congruence relation over terms induced by a given set of equations. This is fundamental in automated reasoning and term rewriting systems.

We first give a formal description of the problem:

- **Signature** Σ: A set of function symbols F, and predicate symbols P, each with an associated arity.
- **Terms** T: The set of all terms constructed from Σ and a set of variables, using the usual rules of term formation.
- **Equations** E: A finite set of equations $s_i = t_i \mid 1 \leq i \leq m$ between terms in T.
- **Congruence Relation**: An equivalence relation that is compatible with the function and predicate symbols in Σ; that is, if $s_i \equiv t_i$ for $1 \leq i \leq k$, then $f(s_1, \ldots, s_k) \equiv f(t_1, \ldots, t_k)$ for any $f \in F$ and $p(s_1, \ldots, s_k) \equiv p(t_1, \ldots, t_k)$ for any $p \in P$.

The goal of the procedure is to compute the smallest congruence relation $\equiv$ over $\mathcal{T}$ such that all equations in E are satisfied: $s_i \equiv t_i$ for all $1 \leq i \leq m$.

The algorithm proceeds in three main phases: Initialization, Processing Input Equations, and Enforcing Congruence Closure. We will describe each phase and analyze its time complexity.

Initialization

- For each term $t \in T$, create a singleton set t in the union-find data structure.
- Initialize an empty **signature table** $\mathcal{S}$, typically implemented as a hash table, to map signatures to representative terms.

Time Complexity $\mathcal{O}(n)$, where $n = |T|$.

Processing Input Equations

For each equation $s = t$ in E:

- Perform $\texttt{union}(\texttt{find}(s), \texttt{find}(t))$ to merge the sets containing s and t.

Time Complexity $\mathcal{O}(m \cdot \alpha(n))$, where $m = |E|$.

Enforcing Congruence Closure

- Repeat until no new unions are performed:
  - For each compound term $f(t_1, t_2, \ldots, t_k) \in T$:
    - Let $r_i = \texttt{find}(t_i)$ for $1 \leq i \leq k$.
    - Construct the *signature* $\sigma = (f, r_1, r_2, \ldots, r_k)$.
    - If σ is in $\mathcal{S}$ with associated term s:
      - Perform $\texttt{union}(\texttt{find}(f(t_1, \ldots, t_k))), \texttt{find}(s))$.
    - Else:
      - Add σ to $\mathcal{S}$ with associated term $f(t_1, \ldots, t_k)$.

Time Complexity We need to consider the needed number of iterations and the time complexity per iterations.

For the time complexity per iteration, finding the representatives for each compound term is in $\mathcal{O}(k \cdot \alpha(n))$, where k is the maximum arity of functions and predicates in Σ. As we consider constant-time hash table operations, constructing the signature and looking it up in $\mathcal{S}$ is in $\mathcal{O}(1)$. The union operation takes $\mathcal{O}(\alpha(n))$ time. Thus, the total time per iteration is in $\mathcal{O}(k \cdot \alpha(n))$.

For the number of iterations, observe that each `union` operation reduces the number of distinct equivalence classes. Since there are at most n terms, there can be at most $n - 1$ unions. However, the structure of the terms may cause the same unions to be considered multiple times. We claim that the number of iterations is bounded by $\mathcal{O}(\log n)$.

We claim that, with the mentioned optimizations, the maximum rank $R_{\max}$ in a union-find data structure with n elements is $\lfloor \log_2 n \rfloor$, which we will show using the following Lemma:

Lemma 7.1. *In the union-find data structure with union by rank, the minimum number of nodes in a tree of rank r is 2^r.*

Proof. We prove this by an inductive argument, considering that a tree's rank can only increase due to `union` operations.

Base case: For $r = 0$, the tree is a singleton node, and the minimum number of nodes is $1 = 2^0$.

Inductive step: Let the rank of a tree be the maximum rank of any node in that tree. When two trees of rank r_1 and r_2 are united and $r_1 \neq r_2$, the resulting tree has rank $\max(r_1, r_2)$ due to the union by rank heuristic and thus contains at least $2^{\max(r_1, r_2)}$ nodes.

Whenever two trees of the same rank r are united, the resulting tree has rank $r + 1$, as one tree is attached to the other ones root. Thus, the resulting tree has at least $2^r + 2^r = 2^{r+1}$ nodes by the induction hypothesis. □

Observe that path compression will only reduce the height of a node's subtree, and thus has no effect on the presented lower bound.

From Lemma 7.1, we immediately obtain the following corollary:

Corollary 7.2. *The maximum rank $R_{\max}$ of any node in the union-find data structure with union by rank is at most $\lfloor \log_2 n \rfloor$.*

Lemma 7.3. *The maximum number of times the representative (root) of any term can change due to* `union` *operations is bounded by $R_{\max}$.*

Proof. Each time a `union` operation changes the parent of a root node, it is because two trees are being merged (non-root nodes are unaffected by `union`). The rank of a root node increases only when two roots of equal rank are united, and as the maximum rank that can occur is $R_{\max}$, the `union` operation can change the representative of any term at most $R_{\max}$ times. □

According to Lemma 7.3, each term can involved in at most $R_{\max}$ union operations that change its rank. Therefore, all terms have stabilized with respect to their parent nodes after at most $R_{\max} \in \mathcal{O}(\log n)$ iterations.

7.3 Complexity Analysis

Combining the complexities of the three phases of the algorithm, we obtain the overall time complexity:

$$\mathcal{O}(n) + \mathcal{O}(m \cdot \alpha(n)) + \mathcal{O}(n \cdot k \cdot \alpha(n) \log n) = \mathcal{O}(n \cdot k \cdot \alpha(n) \log n)$$

Observe that the term k can be considered constant for a fixed signature Σ. As the inverse Ackermann function $\alpha(n)$ grows extremely slowly with n it can be considered a constant for all practical purposes. This gives us the final result of a runtime of $\mathcal{O}(n \log n)$ for the entire algorithm.

8 Homework 9

8.1 Formulate QBF as a first-order theory

We define the theory T_{QBF} over the signature $\langle \{t^0, f^0\}, \{v^1\} \rangle$ with constant function symbols t and f and a unary predicate v.

8.1.1 Define a first-order theory T_{QBF}

We axiomatize the theory:

1. $\forall x, x = t \vee x = f$
2. $t \neq f$
3. $v(t)$
4. $\neg v(f)$

The first two axioms restrict the models to exactly those models with a domain of cardinality of two. The third and fourth axiom ensure that the constant t is interpreted as true and f as false.

8.1.2 Define a function f from closed QBF formulas to T_{QBF} formulas s.t. a formula ϕ is true iff $f(\phi)$ is satisfiable.

Let $\mathcal{P}$ be the set of propositions. We define a function $f : \mathcal{P} \to \mathcal{A}$ from propositions to atomic formulas. We consider formulas as defined in the lecture and add existential quantification and $\top$ as they are used in QBF.

- $f(\top) \mapsto \top$
- $f(\bot) \mapsto \bot$
- $f(p) \mapsto v(p)$

We then get a function $\mathcal{F} : \psi_{QBF} \to \psi_{T_{QBF}}$ by mapping f over the connectives of FOL.

Lemma 8.1. *For any closed QBF formula ϕ, ϕ is true $\iff$ $\mathcal{F}(\phi)$ is satisfiable.*

Proof. We will consider the natural structure $\mathcal{I} \coloneqq \langle \{0,1\}, \{t,f\}, \{v\} \rangle$ with $t_{\mathcal{I}} = 1$, $f_{\mathcal{I}} = 0$ and $v = \{1\}$. We omit writing the subscript $\mathcal{I}$ in the following. Clearly, the structure is a T_{QBF}-*Model*. We prove that ϕ is true *iff* $\mathcal{I} \models \mathcal{F}(\phi)$ by induction on the structure of ϕ.

- $\phi \coloneqq \bot$. For both directions we get an immediate contradiction as $\mathcal{I} \not\models \bot$ and $\bot$ is not true.
- $\phi \coloneqq \top$, trivially $\mathcal{I} \models \mathcal{F}(\top) = \top$ and $\top$ is true, hence $\top$ is true *iff* $\mathcal{I} \models \mathcal{F}(\top)$.
- $\phi \coloneqq p$, (for completenes, ϕ is closed)
- $\phi \coloneqq \phi_1 \Rightarrow \phi_2$. Assume ϕ is true, then either ϕ_1 is false or both ϕ_1 and ϕ_2 are true. By induction, in the first case we get $\mathcal{I} \not\models \mathcal{F}(\phi_1)$, and therefore $\mathcal{I} \models \mathcal{F}(\phi)$ and in the second case $\mathcal{I} \models \mathcal{F}(\phi_1)$ and $\mathcal{I} \models \mathcal{F}(\phi_2)$, hence $\mathcal{I} \models \mathcal{F}(\phi)$. For the other direction, assume $\mathcal{I} \models \mathcal{F}(\phi)$. Then either $\mathcal{I} \not\models \mathcal{F}(\phi_1)$ or both $\mathcal{I} \models \mathcal{F}(\phi_1)$ and $\mathcal{I} \models \mathcal{F}(\phi_2)$. By induction, in the first case ϕ_1 is false, hence ϕ true and in the second case both ϕ_1 and ϕ_2 are true, hence ϕ is true.
- $\phi \coloneqq \forall \phi_1$. By (IH) we know that $\phi_1[\top]$ is true *iff* $\mathcal{I} \models \mathcal{F}(\phi_1[\top])$ and $\phi_1[\bot]$ is true *iff* $\mathcal{I} \models \mathcal{F}(\phi_1[\bot])$. We continute to show that $\mathcal{I} \models \mathcal{F}(\phi_1[\top])$ *iff* $\mathcal{I}_{x \leftarrow 1} \models \mathcal{F}(\phi_1)$ and $\mathcal{I} \models \mathcal{F}(\phi_1[\bot])$ *iff* $\mathcal{I}_{x \leftarrow 0} \models \mathcal{F}(\phi_1)$, hence by definition of $\forall$, $\mathcal{I} \models \mathcal{F}(\phi)$

  case $\top$: Since $\mathcal{I} \models \top \iff v(t)$, we know $\mathcal{I} \models \mathcal{F}(\phi_1[v(t)])$ *iff* $\mathcal{I} \models \mathcal{F}(\phi_1[\top])$ (*Replacement theorem* would require another (trivial) proof by induction on formula). Moreover, $\mathcal{I} \models \mathcal{F}(\phi_1[v(t)]) \iff \mathcal{I}_{x \leftarrow 1} \models \mathcal{F}(\phi_1[v(x)])$ and $\phi_1[v(x)] = \phi_1$. By transitivity of $\iff$, $\phi_1[\top]$ is true *iff* $\mathcal{I}_{x \leftarrow 1} \models \mathcal{F}(\phi_1)$.

case $\bot$: Symmetric to **case** $\top$.

- $\phi \coloneqq \exists\phi_1$. By definition, either $\phi_1[\bot]$ or $\phi_1[\top]$ is true. We proceed by cases, by (IH) we know that in the first case $\phi_1[\top]$ is true *iff* $\mathcal{I} \models \mathcal{F}(\phi_1[\top])$ and in the other case $\phi_1[\bot]$ is true *iff* $\mathcal{I} \models \mathcal{F}(\phi_1[\bot])$. For each case, the proof is identical to the cases $\bot$ and $\top$ above.

To prove the final statement, assume ϕ is true, then by the proof above $\mathcal{F}(\phi)$ is satisfiable as $\mathcal{I} \models \mathcal{F}(\phi)$. For the other direction, assume $\mathcal{F}(\phi)$ is satisfiable, then there exists an $\mathcal{I}'$ s.t. $\mathcal{I}' \models \mathcal{F}(\phi)$. By the axioms, $\mathcal{I}'$ is isomorphic to $\mathcal{I}$, since $\mathcal{I}'$ is a T_{QBF}-Model and therefore by axioms (1) and (2) has a 2 element domain, with the element $x = [\![t]\!]'_{\mathcal{I}}$ being the only element s.t. $x \in [\![v]\!]'_{\mathcal{I}}$. Thereofre for any ψ, $\mathcal{I}' \models \psi$ *iff* $\mathcal{I} \models \psi$. Hence, also $\mathcal{I} \models \mathcal{F}(\phi)$ and therefore ϕ is true. □

8.2 Argue that Pressburger arithmetic does not allow QE.

The goal of this exercise is to show that Presburger arithmetic $\mathcal{T}_{PR}$ as defined in the lecture (i.e., without divisibility predicate) does *not* allow for quantifier elimination (QE).

It is important to point out that in most cases, Presburger arithmetic is considered with the divisibility predicate, which allows for quantifier elimination, this construction was even given by Presburger in his original 1929 paper.

To show that QE does not work without the divisibility predicate, consider the $\mathcal{T}_{PR}$ formula φ that expresses that a variable y is even.

$$\varphi: \quad \exists x.\ x + x \leq y \wedge y \leq x + x$$

Note that in this formula, y is a *free variable*. Depending on the value of y, the formula φ is either true or false. The set of values of y for which φ is true is the set of even numbers, which is countably infinite.

For the sake of readability, we say a formula ψ with a single free variable x *defines* a subset of the natural numbers S if ψ evaluates to true if and only if $x \in S$. We now claim that any *quantifier-free* $\mathcal{T}_{PR}$-formula ψ with a single free variable defines a subset of the natural numbers that can be expressed as a finite union of intervals. To show this, we first consider atomic formulas and then proceed by induction on the structure of ψ.

Lemma 8.2. *Every quantifier-free, atomic $\mathcal{T}_{PR}$-formula ψ with a single free variable x defines a contiguous subset of the natural numbers.*

Proof. Denoting repeated addition of x by ax using some constant a, we can express any atomic formula in $\mathcal{T}_{PR}$ as $ax + b \leq cx + d$, which simplifies to $(a - c)x \leq d - b$. This atomic formula is true for integer values of x in the following intervals:

- When $a > c$:
  - If $d \geq b$, then $x \in \left[0, \frac{d-b}{a-c}\right]$.

- If $d < b$, then $x \in \left[\frac{d-b}{a-c}, \infty\right)$. However, since $\frac{d-b}{a-c}$ is negative and $x \in \mathbb{N}$, this effectively means $x \in [0, \infty)$.

- When $a < c$:
  - If $d \geq b$, then $x \in \left[\frac{d-b}{a-c}, \infty\right)$.
  - If $d < b$, then $x \in \left[0, \frac{d-b}{a-c}\right]$.

- When $a = c$:
  - If $d \geq b$, then $x \in \mathbb{N}$.
  - If $d < b$, the formula is false, i.e. x is in the empty interval.

□

Theorem 8.3. *Any quantifier-free $\mathcal{T}_{PR}$-formula ψ with a single free variable x defines a finite union of intervals.*

Proof. We proceed by induction on the structure of ψ.

Base case: By Lemma 8.2, if ψ is an atomic formula, it defines a contiguous subset of the natural numbers (i.e., a single interval). **Induction step**: Let ψ be a boolean combination of formulas φ_1 and φ_2, that both describe finite unions of intervals.

- $\psi := \neg\varphi_1$: This describes the complement of the intervals defined by φ_1. The complement of a finite union of intervals is a finite union of intervals.

- $\psi := \varphi_1 \wedge \varphi_2$: This describes the intersections of the intervals defined by φ_1 and φ_2. The intersection of two finite unions of intervals is a finite union of intervals.

- $\psi := \varphi_1 \vee \varphi_2$: This describes the union of the intervals defined by φ_1 and φ_2. The union of two finite unions of intervals is still a finite union of intervals.

□

Observe that every finite union of intervals can only describe a subset of the natural numbers of a certain structure:

- A finite set of natural numbers, if the union does not contain the interval $[k, \infty)$ for some $k \in \mathbb{Q}^+ \cup \{0\}$.

- Otherwise, the union of a finite set of natural numbers and $\{n \mid n \in \mathbb{N} \wedge n \geq k\}$ for some $k \in \mathbb{Q}^+ \cup \{0\}$.

It is easy to see that thus, every subset of natural numbers defined by a finite union of intervals is either *finite* or *has a finite complement* (when it contains $[k, \infty)$ for some k). This is in contrast to the definition of any periodic set of natural numbers like the even numbers, which is both countably infinite and has a countably infinite complement. Therefore, any periodic set of natural numbers cannot be described by a quantifier-free $\mathcal{T}_{PR}$-formula.

9 Homework 10: Untyped Lambda Calculus

This homework contains two exercises on the untyped lambda calculus. For readability, we will use the following definitions from the lecture:

$$\begin{array}{lll} \mathrm{I} = \lambda x.x & \top = \lambda xy.x & \bot = \lambda xy.y \\ \mathrm{if} = \lambda bxy.bxy & \underline{0} = \lambda Sx.x & \underline{1} = \lambda Sx.Sx \\ \underline{2} = \lambda Sx.S(Sx) & \underline{n} = \lambda Sx.S^n(x) & \mathrm{add} = \lambda mn.mSn \\ \mathrm{mult} = \lambda mn.m(\mathrm{add}\, n)\, \underline{0} & \mathrm{iszero} = \lambda n.n(\lambda x.\bot)\top & Y = \lambda f.(\lambda x.fxx)(\lambda x.fxx) \end{array}$$

9.1 Predecessor function for Church numerals

$$\mathrm{pred} = \lambda nfx.n(\lambda gh.h(gf))(\lambda u.x)(\lambda u.u)$$

The predecessor function pred should take a numeral n and return $n-1$. We know that:

- $\mathrm{pred}(\underline{0}) = \underline{0}$, since the predecessor of zero should not go below zero.
- For any $n > 0$, $\mathrm{pred}(n) = n - 1$.

A clever technique to define pred uses a "pairing" function that keeps track of two values as we iterate through the numeral. The idea is to transform the iteration function of the numeral into one that carries along a state representing the current and previous "count". By the time we finish applying the numeral's function the appropriate number of times, we end up with the predecessor.

We want an iteration that, given a function f and an initial value x, produces something related to $(n-1)$. Consider we apply n times a function that takes a pair (g, h) and returns a new pair that effectively shifts f from "next" to "previous":

1. Start with a pair:
   - A "dummy" function $\lambda u.x$ that, if applied, yields x.
   - The identity function $\mathrm{I} = \lambda u.u$ which essentially acts as the successor chain.

   We think of this initial pair as $(\lambda u.x, \lambda u.u)$. The idea is that the second element of the pair eventually ends up "one step ahead" of the first. After iterating n times, the first component of the pair will encode the predecessor computation.

2. Each iteration step transforms (g, h) into $(h, \lambda z.h(gf))$ or something equivalent. More concretely, we define a function $\lambda gh.h(gf)$ that, given a pair (g, h), produces a new function related to this shifting. By repeatedly applying this, we "shift" through the functions, so that after n steps, the "front" shifts out one step ahead, and what remains in the first component is effectively the predecessor function applied to f and x.

This yields the following lambda term for pred:

$$\text{pred} = \lambda nfx.n(\lambda gh.h(gf))(\lambda u.x)(\lambda u.u).$$

We illustrate the correctness of this definition by β-reducing $\text{pred}\,\underline{0}$ and $\text{pred}\,\underline{2}$:

Reduction of $\text{pred}\,\underline{0}$

$$\begin{aligned}
\text{pred}\,\underline{0} &= (\lambda nfx.n(\lambda gh.h(gf))(\lambda u.x)(\lambda u.u))\,\lambda Sx'.x' \\
&\to_\beta \lambda fx.(\lambda Sx'.x')(\lambda gh.h(gf))(\lambda u.x)(\lambda u.u) \\
&\to_\beta \lambda fx.(\lambda x'.x')(\lambda u.x)(\lambda u.u) \\
&\to_\beta \lambda fx.(\lambda u.x)(\lambda u.u) \\
&\to_\beta \lambda fx.x \\
&= \underline{0}
\end{aligned}$$

Reduction of $\text{pred}\,\underline{2}$

$$\begin{aligned}
\text{pred}\,\underline{2} &= (\lambda nfx.n(\lambda gh.h(gf))(\lambda u.x)(\lambda u.u))\,(\lambda Sx.S(Sx)) \\
&\to_\beta \lambda fx.(\lambda Sx'.S(Sx'))(\lambda gh.h(gf))(\lambda u.x)(\lambda u.u) \\
&\to_\beta \lambda fx.(\lambda x'.(\lambda gh.h(gf))((\lambda gh.h(gf))x'))(\lambda u.x)(\lambda u.u) \\
&\to_\beta \lambda fx.(\lambda gh.h(gf))((\lambda gh.h(gf))(\lambda u.x))(\lambda u.u) \\
&\to_\beta \lambda fx.(\lambda gh.h(gf))(\lambda h.h(x))(\lambda u.u) \\
&\to_\beta \lambda fx.(\lambda h.h((\lambda h.h(x))f))(\lambda u.u) \\
&\to_\beta \lambda fx.(\lambda h.h(f(x)))(\lambda u.u) \\
&\to_\beta \lambda fx.f(x) \\
&= \underline{1}
\end{aligned}$$

9.2 β-Reduction of fact 2

For the sake of readability, we assume that all arithmetic functions defined in the lecture correspond to their intended behavior (i.e., we assume that add $nm = n + m$, mult $nm = n \cdot m$, and iszero $n = \top$ if $n = 0$ and $\bot$ otherwise). Steps following from these definitions are written using a double arrow $\Rightarrow_\beta$.

$$\begin{aligned}
\text{fact}\,\underline{2} &= (Y(\lambda fn.\text{if}(\text{iszero}\,n)\underline{1}(\text{mult}\,n(f(\text{pred}\,n)))))\,\underline{2} \\
&= ((\lambda f.f(\lambda x.fxx)(\lambda x.fxx))(\lambda fn.\text{if}(\text{iszero}\,n)\underline{1}(\text{mult}\,n(f(\text{pred}\,n)))))\,\underline{2} \\
&\to_\beta (\lambda x.(\lambda fn.\text{if}(\text{iszero}\,n)\underline{1}(\text{mult}\,n(f(\text{pred}\,n))))(xx)) \\
&\quad (\lambda x.(\lambda fn.\text{if}(\text{iszero}\,n)\underline{1}(\text{mult}\,n(f(\text{pred}\,n))))(xx))\,\underline{2} \\
&\to_\beta (\lambda n.\text{if}(\text{iszero}\,n)\underline{1}(\text{mult}\,n((Y(\lambda fn.\text{if}(\text{iszero}\,n)\underline{1}(\text{mult}\,n(f(\text{pred}\,n)))))(\text{pred}\,n))))\,\underline{2} \\
&\to_\beta \text{if}(\text{iszero}\,\underline{2})\underline{1}(\text{mult}\,\underline{2}(\text{fact}(\text{pred}\,\underline{2}))) \\
&\to_\beta \text{if}(\bot)\underline{1}(\text{mult}\,\underline{2}(\text{fact}\,\underline{1})) \\
&\to_\beta \text{mult}\,\underline{2}(\text{fact}\,\underline{1})
\end{aligned}$$

The first two reductions of computing fact $\underline{1}$ and fact $\underline{0}$ are analogous to the previous computation and will thus be omitted.

$$\begin{aligned}
\text{fact}\,\underline{1} &= (Y(\lambda fn.\text{if}(\text{iszero}\,n)\underline{1}(\text{mult}\,n(f(\text{pred}\,n)))))\,\underline{1} \\
&\Rightarrow_\beta \text{if}(\text{iszero}\,\underline{1})\underline{1}(\text{mult}\,\underline{1}(\text{fact}(\text{pred}\,\underline{1}))) \\
&\to_\beta \text{if}(\bot)\underline{1}(\text{mult}\,\underline{1}(\text{fact}\,\underline{0})) \\
&\to_\beta (\bot)\underline{1}(\text{mult}\,\underline{1}(\text{fact}\,\underline{0})) \\
&\to_\beta \text{mult}\,\underline{1}(\text{fact}\,\underline{0})
\end{aligned}$$

$$\begin{aligned}
\text{fact}\,\underline{0} &= (Y(\lambda fn.\text{if}(\text{iszero}\,n)\underline{1}(\text{mult}\,n(f(\text{pred}\,n)))))\,\underline{0} \\
&\Rightarrow_\beta \text{if}(\text{iszero}\,\underline{0})\underline{1}(\text{mult}\,\underline{0}(\text{fact}(\text{pred}\,\underline{0}))) \\
&\to_\beta \text{if}(\top)\underline{1}(\text{mult}\,\underline{0}(\text{fact}\,\underline{0})) \\
&\to_\beta (\top)\underline{1}(\text{mult}\,\underline{0}(\text{fact}\,\underline{0})) \\
&\to_\beta \underline{1}
\end{aligned}$$

From these computations, we can conclude that $\text{fact}\,\underline{2} = \text{mult}\,\underline{2}(\text{fact}\,\underline{1}) = \text{mult}\,\underline{2}(\text{mult}\,\underline{1}(\text{fact}\,\underline{0})) = \text{mult}\,\underline{2}(\text{mult}\,\underline{1}\,\underline{1}) = \text{mult}\,\underline{2}\,\underline{1} = \underline{2}$.

10 Homework 15

Consider the following formulae:

$$\varphi_1 := \exists\Box\exists\Diamond p$$

$$\varphi_2 := \forall\Box\forall\Diamond p$$

Let us add two new primitives to CTL:
$\psi_1 := \exists\Box\Diamond p$ – there is a path along which p is true infinitely often
$\psi_2 := \forall\Box\Diamond p$ – for all paths, p is true infinitely often
Define the two new primitives in μ-calculus and (dis)prove whether $\psi_1 \Rightarrow \varphi_1, \varphi_1 \Rightarrow \psi_1, \psi_2 \Rightarrow \varphi_2, \varphi_2 \Rightarrow \psi_2$

Proof. Let us define ψ_1 and ψ_2 in μ-calculus.

$$\psi_1 := \nu x.(\mu y.(p \wedge \exists \bigcirc x) \vee \exists \bigcirc y)$$

$$\psi_2 := \nu x.(\mu y.(p \wedge \forall \bigcirc x) \vee \forall \bigcirc y)$$

Let us also express φ_1 and φ_2 in μ-calculus.

$$\varphi_1 := \nu x.(\mu y.(p \vee \exists \bigcirc y) \wedge \exists \bigcirc x)$$

$$\varphi_2 := \nu x.(\mu y.(p \vee \forall \bigcirc y) \wedge \forall \bigcirc x)$$

Intuitively, φ_1 means that there exists a path along which p is always reachable. Dually, φ_2 means that for all paths, it is always true that p is reached by taking any path.

First, we prove that $\psi_1 \Rightarrow \varphi_1$. Let $s = s_1 s_2 \cdots$ be a path along which p is true infinitely often. Then, along this path, p is always reachable as we might just continue as s goes and we reach p.

Let us now prove that $\varphi_1 \not\Rightarrow \psi_1$. Let us define a Kripke structure $(\{s_1, s_2, s_3\}, \rightarrow, [\![\cdot]\!])$ where $\rightarrow$ is $\{(s_1, s_1), (s_1, s_2), (s_2, s_3), (s_3, s_3)\}$ and $[\![s_1]\!] = [\![s_3]\!] = \emptyset$ and $[\![s_2]\!] = \{p\}$. Then, state s_1 satisfies φ_1 as there exists a path $s_1 s_1 s_1 \cdots$ along which p is always reachable (as we can just move to s_2 and then loop at s_3). However, s_1 does not satisfy ψ_1 as there is no path along which p is true infinitely often.

Next, we prove $\psi_1 \Rightarrow \varphi_2$. Let us take any state on any path and any path going from this state. We shall prove that p is eventually reached. However, this is a direct consequence of ψ_1 as p is visited infinitely often no matter what path we take.

Finally, let us prove that $\neg\varphi_2 \Rightarrow \neg\psi_2$ which is equivalent to $\varphi_2 \Rightarrow \psi_2$. Let us take the witness of the contradiction, i.e., let s be a path and q be a state on the path s and let s' be a path from q such that p is not reached on s'. This violates that p is true infinitely often on any path as we might just take s and then switch to s' and p is visited only finitely many times. □

11 Homework 17: Show that L(CCS) is not necessarily context-free

We prove the claim by constructing a CCS process P s.t. $L(P) = \{a^m b^n c^k \mid m \geq n \geq k\}$, which we prove non context-free. We use the convention of not including τ in the language of a process.

11.1 Prove language non context-free

Consider the language $\mathcal{L} = \{a^m b^n c^k \mid m \geq n \geq k\}$. We prove that the language is not context-free using the pumping lemma for context-free grammars (Adapted the proof of $\{a^m b^n c^k \mid m > n > k\}$ not context-free[1]).

Assume towards a contradiction that $\mathcal{L}$ is context-free, then by the pumping lemma there must be some pumping constant p s.t. for any word z with $|z| \geq p$, there exist some w, u, v, x, y s.t. $z = uvwxy$ and $|vx| > 0$, $|vwx| \leq p$, and $n \forall n. uv^n wx^n y \in \mathcal{L}$.

Take $z = a^p b^p c^p$, then there must be some decomposition $uvwxy = z$ with the properties stated above. We consider two cases.

a) Assume vwx contains no a. Then $uv^2wx^2y \notin \mathcal{L}$ as it contains either more bs or more cs than as.

b) Assume vwx contains some a, then it cannot contain any cs as $|vwx| < p$ and the as and cs are separated by p bs. Now, consider the word $uwy = uv^0wx^0y \in \mathcal{L}$. It contains p cs but at most $2p-1$ as and bs (since $|vx| > 0$). But then it cannot be the case that both $\#a \geq \#c$ and $\#b \geq \#c$, hence we reached a contradiction.

11.2 Construct P

We now construct the CCS process.

$$
\begin{aligned}
A &\coloneqq (aA \parallel !s_a) + !s_1 \\
B &\coloneqq (?s_a bB \parallel !s_b) + !s_2 \\
C &\coloneqq (?s_b c \parallel C) \\
P &\coloneqq \nu s_a s_b s_1 s_2 . A \parallel (?s_1 B) \parallel (?s_2 C)
\end{aligned}
$$

Intuitively, the process hides all symbols other than a, b, and c and use choice $(+)$ to enforce the correct order of symbols and synchronization to enforce the inequalities.

Proof. We give an informal argument to argue that $L(P) = \mathcal{L}$.

$\Rightarrow$ Assume $w \in L(P)$, we show $w \in \mathcal{L}$. If $w \in L(P)$, then there is some trace $p_0 \xrightarrow{s_0} p_1 \xrightarrow{s_1} \cdots \xrightarrow{s_j} p_{j+1}$ and $s_0 s_1 \ldots s_j = w$.

Assume towards a contradiction that there is some $s_j = a$ and $s_i = b$ with $i > j$. Note that b can only be sent by subprocess B, which is guarded by a synchronization with s_1. Hence, this can only be the case if subprocess A sends s_1 on some index $j < o < i$. By construction of A this is not possible as s_1 ends the recursion which is guarded with as. A similar argument applies for b and c, hence enforcing the correct order. Now we reason about the inequalities. By construction of subprocess A, $\#a = \#!s_a$, and since $\#?s_a = \#b$, we get $\#a \geq \#b$ as each b is guarded by a synchronization between s_a. Similar argument for $\#b \geq \#c$.

[1] http://infolab.stanford.edu/ ullman/ialcsols/sol7.html

$\Leftarrow$ Assume $w \in \mathcal{L}$, we show $w \in L(P)$. Let n, m, k s.t. $w = a^n b^m c^k$, we show that there is a trace producing this word (For simplicity we don't write the hiding).

We first recurse in the A subprocess to generate m as and by doing so, generate also m $!s_a$ in parallel (step 1). Then, we take the right choice of A, synchronizing with s_1. Now we unroll the recursion of B by synchronizing on s_a and sending b. Note, that since $m \geq n$ we can do this n times. And now we continue in a similar fashion for C.

$$P \xrightarrow{a^m}{}^* ((aA \parallel !s_a) + !s_1) \parallel \overbrace{!s_a \cdots !s_a}^{m} \parallel (?s_1 B) \parallel (?s_2 C) \xrightarrow{\tau} \overbrace{!s_a \cdots !s_a}^{m} \parallel B \parallel (?s_2 C) \xrightarrow{\tau}$$
$$\overbrace{!s_a \cdots !s_a}^{m-1} \parallel bB \parallel !s_b \parallel (?s_2 C) \xrightarrow{(\tau b)^n}{}^*$$
$$\overbrace{!s_a \cdots !s_a}^{m-n} \parallel !s_2 \parallel \overbrace{!s_b \cdots !s_b}^{n} \parallel (?s_2 C) \xrightarrow{\tau} \overbrace{!s_a \cdots !s_a}^{m-n} \parallel C \xrightarrow{(\tau c)^k}{}^*$$
$$\overbrace{!s_a \cdots !s_a}^{m-n} \parallel \overbrace{!s_b \cdots !s_b}^{n-k} \parallel C$$

□